

如何解决LLM致幻— GPT

这是很多做 **RAG + Multi-Agent 股票分析系统** 的团队都会遇到的问题。

你现在的体系本质上是：

```
graph TD; A[用户提问] --> B[Retriever]; B --> C[已有股票分析文档]; C --> D[LLM分析]; D --> E[输出结果];
```

对于已经分析过的股票：

```
graph TD; A[TSLA] --> B[向量库找到大量历史分析]; B --> C[LLM基于真实资料回答]; C --> D[效果很好];
```

但对于第一次分析的股票：

```
graph TD; A[新股票XYZ] --> B[向量库为空]; B --> C[Retriever返回空]; C --> D[LLM只能靠预训练知识]; D --> E[开始胡编];
```

这实际上不是幻觉问题，而是 **Cold Start（冷启动）** 问题。

第一层：建立 Online Research Agent

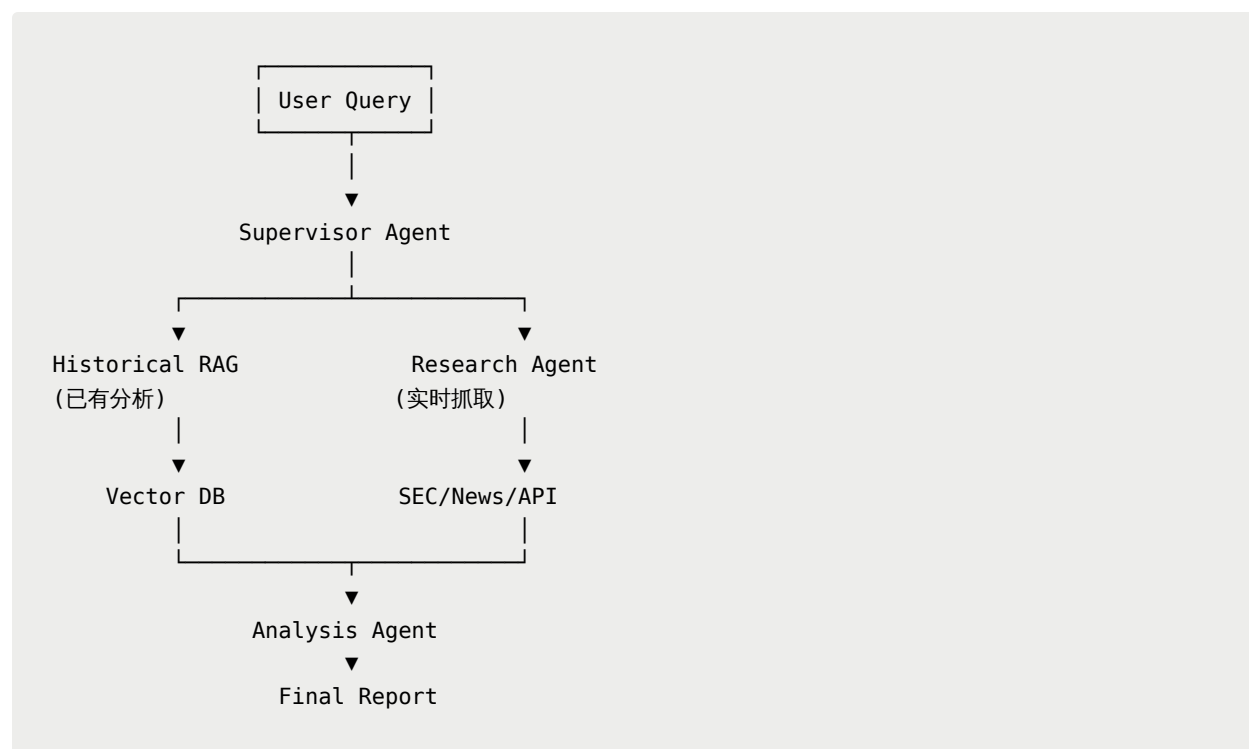
很多人误以为：

向量库 = 知识来源

其实应该是：

向量库 + 实时数据源

架构应该改成：



例如：

用户问：

分析 PLTR

如果发现：

```
vector search score < threshold
```

说明：

数据库没有资料

触发：

Research Agent

去获取：

- 公司简介
- 财报
- 新闻
- 分析师评级
- 财务指标

然后再分析。

这样LLM就不会裸奔。

第二层：先生成 Research Packet

不要让LLM边搜边分析。

很多系统这样做：

```
Search Agent  
↓  
找到网页  
↓  
直接交给LLM  
↓  
分析
```

效果很差。

更好的做法：

```
Research Agent
↓
收集信息
↓
结构化整理
↓
生成 Research Packet
↓
Analysis Agent
```

例如：

```
{
  "ticker": "PLTR",
  "company_profile": "...",
  "latest_earnings": "...",
  "revenue_growth": "29%",
  "pe_ratio": "110",
  "analyst_rating": "Buy",
  "recent_news": [
    "...",
    "..."
  ]
}
```

然后：

```
Analysis Agent
```

只允许依据：

Research Packet

分析。

这样幻觉会下降很多。

第三层：强制 Evidence-Based Analysis

很多股票Agent最大的bug：

Agent拿到资料
↓
开始脑补

例如：

Palantir未来一定会成为AI时代最大赢家

这根本没有证据。

你应该要求：

每个结论必须引用证据

Prompt：

Rules:

1. Every claim must cite evidence.
2. If evidence is missing, say "Insufficient data".
3. Never infer financial metrics.
4. Never fabricate analyst ratings.

输出：

Bull Case

Claim:
Revenue growth remains strong.

Evidence:
Q1 revenue +29% YoY.

Confidence:
High

而不是：

Palantir将持续高速增长

第四层：加入 Confidence Agent

这是很多面试官会喜欢的设计。

增加一个专门Agent：

Confidence Agent

职责：

评估当前分析可信度

例如：

```
{  
  "data_coverage":87,  
  "news_count":12,  
  "financial_metrics":25,  
  "confidence":"HIGH"  
}
```

如果：

```
{  
  "data_coverage":20,  
  "confidence":"LOW"  
}
```

最终报告显示：

```
△ Low Confidence Analysis  
  
Reason:  
Limited information available.
```

而不是输出一个看起来很专业但全是猜测的报告。

第五层：把第一次分析变成知识沉淀

第一次分析最贵。

不要浪费。

流程：

```
第一次分析  
↓  
Research Agent  
↓  
Analysis Agent  
↓  
Final Report  
↓  
Knowledge Extraction Agent  
↓  
Vector DB
```

变成：

```
第二次分析同股票  
↓
```

直接RAG召回

↓

成本下降90%

这才是股票Agent真正的飞轮。

第六层：引入 Planner Agent（高级版）

很多股票分析失败是因为：

直接分析

但人类分析师会先列框架。

例如：

分析 NVDA

Planner Agent先拆解：

```
[  
  "Company Overview",  
  "Financial Analysis",  
  "Valuation",  
  "Competitive Position",  
  "Risk Factors",  
  "Recent News"  
]
```

然后：

```
Financial Agent  
News Agent  
Valuation Agent  
Risk Agent
```

分别完成。

最后：

Supervisor Agent

汇总。

这就是 LangGraph 特别适合的场景。

如果这是求职项目，我最推荐的升级路线

你现在可能是：

RAG
+
Multi-Agent
+
Hallucination Check

面试官会觉得：

不错，但比较常见

升级成：

Supervisor Agent

├─ RAG Agent
├─ Research Agent
├─ Financial Data Agent
├─ News Agent
├─ Confidence Agent
├─ Knowledge Extraction Agent

最终形成：

Research → Analyze → Verify → Store

然后你可以讲一个非常有价值的设计思想：

“我发现传统 RAG 只能解决已有知识的问题，对新股票存在冷启动缺陷。因此我设计了一个 Hybrid Knowledge Architecture：优先从向量库检索历史分析；若知识覆盖率不足，则自动触发 Research Agent 获取实时数据，生成结构化 Research Packet 后再进行分析，并通过 Confidence Agent 对分析可信度进行量化评分，最后将新知识回写向量库形成持续学习闭环。”

这个回答会比单纯说“我做了个股票分析 Agent”有深度得多，因为它体现了你对 **RAG 冷启动、Agent 协作、知识闭环、幻觉治理** 的理解。